

Universidad
Rey Juan Carlos

Escuela Técnica Superior
de Ingeniería Informática

Grado en Matemáticas

Curso 2025-2026

GEOMETRÍA COMPUTACIONAL

PRÁCTICA 9
TRIANGULACIONES Y COLORACIÓN DE GRAFOS EN R

Autores: David Hernández Paradela, Daniel Rodríguez Fernández

Resumen

Este trabajo aborda dos problemas fundamentales de la Geometría Computacional en el entorno de programación `R`: la triangulación de polígonos arbitrarios y la coloración de grafos. Para la triangulación, se diseña e implementa un algoritmo recursivo basado en la detección de orejas que descompone cualquier polígono simple en triángulos. Para la coloración, se implementa el algoritmo *DSATUR*, que asigna colores a los vértices de un grafo minimizando el número de colores utilizados. Ambos algoritmos se validan sobre una variedad de casos de prueba representativos, incluyendo polígonos convexos, cóncavos y degenerados, así como grafos con distintas estructuras topológicas. Finalmente, se utiliza este último algoritmo para colorear dos mapas reales.

Palabras clave:

- Triangulación de polígonos
- Detección de orejas
- Coloración de grafos
- DSATUR
- $\mathbb{R}^2$
- `R`

Índice de contenidos

1. Introducción **1**

2. Objetivos **3**

3. Parte I: Triangulación de Polígonos **4**
- 3.1. Fundamentos teóricos . . . 4
- 3.2. Descripción del algoritmo . . . 5
- 3.3. Casos de prueba y resultados . . . 6
- 3.4. Consideraciones sobre casos especiales . . . 8

4. Parte II: Coloración de Grafos **9**
- 4.1. Fundamentos teóricos . . . 9
- 4.2. Descripción del algoritmo . . . 10
- 4.3. Grafos de prueba . . . 10
- 4.4. Resultados . . . 10

5. Resultados y Discusión **13**
- 5.1. Triangulación . . . 13
- 5.2. Coloración de grafos . . . 13
- 5.3. Aplicación a un mapa geográfico . . . 14

6. Conclusiones y Trabajos Futuros **17**
- 6.1. Conclusiones . . . 17
- 6.2. Trabajos futuros . . . 17

1 Introducción

La Geometría Computacional es una rama de las ciencias de la computación que estudia el diseño y análisis de algoritmos para resolver problemas de naturaleza geométrica. Entre sus problemas más clásicos y con mayor rango de aplicación se encuentran la **triangulación de polígonos** y la **coloración de grafos**, que son precisamente los dos pilares de esta práctica.

La triangulación de un polígono consiste en descomponerlo en triángulos no solapados cuyos vértices son los del polígono original. Este proceso es fundamental en multitud de aplicaciones, desde la síntesis de gráficos tridimensionales y el mallado de elementos finitos hasta el procesamiento de imágenes y la robótica. Desde el punto de vista teórico, el resultado principal que justifica la viabilidad del proceso es el Teorema de Triangulación de Polígonos [1], que garantiza que todo polígono simple de n vértices admite una triangulación en $n-2$ triángulos mediante la adición de $n-3$ diagonales interiores. Este teorema ha sido estudiado en la parte teórica de la asignatura como parte del Teorema de la Galería de Arte.

La coloración de grafos, por su parte, es un problema de asignación en el que se busca etiquetar los vértices de un grafo con el menor número posible de colores, de forma que ningún par de vértices adyacentes comparta color. Aunque determinar el número cromático óptimo es un problema NP-completo en general, existen heurísticas eficaces que producen coloraciones de calidad en tiempo polinomial. La conexión entre ambos problemas no es casual: el Teorema de la Galería de Arte, vista en la parte teórica de la asignatura, demuestra que una vez triangulado un polígono, la coloración de sus vértices con tres colores proporciona directamente una estrategia óptima para la vigilancia de la galería, por lo que es posible determinar las posiciones óptimas de vigilantes con a lo sumo $\lfloor n/3 \rfloor$

guardias.

Este trabajo presenta la implementación en `R` de un algoritmo de triangulación recursivo basado en la eliminación de orejas (combinado con un algoritmo del tipo Divide y Vencerás) y un algoritmo de coloración de grafos basado en la saturación de vértices (*DSATUR*). Ambos se prueban sobre distintos tipos de entradas para validar su corrección y comportamiento.

2

Objetivos

1. **Diseño e implementación de un algoritmo de triangulación general:** Desarrollar en `R` un algoritmo capaz de triangular cualquier polígono simple, convexo o cóncavo, dado como una secuencia ordenada de vértices en sentido antihorario. El algoritmo debe gestionar correctamente casos degenerados como vértices casi colineales o polígonos tipo peine.

2. **Diseño e implementación de un algoritmo de coloración de grafos:** Implementar en `R` el algoritmo *DSATUR* para la coloración de grafos representados por su lista de aristas. El algoritmo debe minimizar el número de colores empleados y ser aplicable a distintas familias de grafos: ciclos, completos, bipartitos, ruedas y grafos arbitrarios.

3. **Validación experimental de los algoritmos:** Aplicar los algoritmos implementados a una batería de casos de prueba representativos, incluyendo polígonos de distinta morfología y grafos con estructura variada, verificando la corrección de los resultados y midiendo los tiempos de ejecución.

4. **Visualización de los resultados:** Desarrollar funciones de representación gráfica en `R` que permitan visualizar de forma clara las triangulaciones obtenidas y las coloraciones de los grafos, facilitando la interpretación de los resultados.

5. **Aplicación a mapas geográficos reales:** Aplicar el algoritmo DSATUR sobre mapas reales (China y Estados Unidos), codificando la relación de adyacencia entre regiones como un grafo y verificando que la coloración obtenida es óptima y consistente con el Teorema de los Cuatro Colores.

3

Parte I: Triangulación de Polígonos

3.1. Fundamentos teóricos

Una **triangulación** de un polígono P de n vértices es una partición de su interior en triángulos no solapados, cuyos vértices son vértices de P y cuyos lados son lados o diagonales de P. Se puede demostrar que:

- Todo polígono simple posee al menos un vértice **estrictamente convexo**, es decir, un vértice en el que el giro al recorrer el contorno en sentido antihorario es hacia la izquierda.

- El Lema de Meisters[1] garantiza que todo polígono de $n \geq 4$ vértices tiene al menos una **oreja**: un vértice convexo v_i tal que el triángulo formado por v_{i-1}, v_i y v_{i+1} no contiene ningún otro vértice del polígono en su interior. Esto fue también visto en la parte teórica de la asignatura, como ejercicio, cuya demostración era casi directa de ver que el grafo dual de la triangulación de un polígono simple es un árbol.

- Todo polígono de n vértices se puede triangular usando exactamente $n-3$ diagonales, obteniéndose $n-2$ triángulos.

Concepto de oreja

Dado un vértice convexo v_i con vecinos v_{i-1} y v_{i+1}, se dice que v_i es una **oreja** si el segmento $v_{i-1}v_{i+1}$ (la diagonal que lo elimina) está completamente contenido en el interior del polígono, lo que equivale a que ningún otro vértice del polígono caiga dentro del triángulo $\triangle(v_{i-1}, v_i, v_{i+1})$.

Para detectar si un punto p_l cae en el interior estricto de dicho triángulo, se utilizan áreas orientadas: el punto está en el interior si y solo si los tres determinantes $\Delta(v_{i-1}, v_i, p_l)$, $\Delta(v_i, v_{i+1}, p_l)$ y $\Delta(v_{i+1}, v_{i-1}, p_l)$ son todos estrictamente positivos (en un sistema antihorario).

3.2. Descripción del algoritmo

En base a los resultados resumidos previamente, se desarrolla un algoritmo que es una mezcla entre el algoritmo clásico de recorte de orejas (ear-clipping [3]) y el algoritmo visto en clase en la demostración del Teorema de la Galería de Arte, que es de tipo Divide y Vencerás. Su estructura general es la siguiente:

1. Si el polígono actual tiene 3 vértices, devolver el triángulo y terminar.
2. Buscar un vértice convexo v_i en la lista actual de índices.
3. Comprobar si v_i está en una oreja, buscando entre todos los vértices restantes si alguno está contenido en el triángulo:
   - Si lo es: la diagonal es $v_{i-1}v_{i+1}$. Se añade el triángulo $\triangle(v_{i-1}, v_i, v_{i+1})$ y se llama recursivamente sobre el subpolígono resultante de eliminar v_i.
   - Si no lo es: entre los vértices del polígono que caen dentro del triángulo, se elige el más cercano a v_i (en distancia euclídea al cuadrado) como extremo de la diagonal. El polígono se divide en dos subpolígonos sobre los que se recurre de forma independiente.
4. Se devuelve la unión de todas las listas de triángulos producidas por las llamadas recursivas.

Funciones auxiliares implementadas

El código en `R` organiza el algoritmo en varias funciones con responsabilidades bien delimitadas:

- `areaOrientada(p, q, r)`: calcula el determinante de la matriz formada por los vectores $\overrightarrow{pq}$ y $\overrightarrow{pr}$. Devuelve un valor positivo si el giro $p \to q \to r$ es antihorario, negativo si es horario, y cero si los puntos son colineales. Es la primitiva geométrica fundamental del algoritmo.
- `convexo(Poli, indices, i)`: determina si el vértice i-ésimo (en el subpolígono definido por `indices`) es convexo, comprobando que `areaOrientada` sea positiva con tolerancia numérica.
- `enTriangulo(Poli, indices, i, l)`: comprueba si el vértice l-ésimo está en el interior estricto del triángulo asociado a la oreja candidata i.
- `esOreja(Poli, indices, i)`: dado un vértice convexo i, determina si es una oreja. Si no lo es, devuelve el índice del vértice interior más cercano a v_i, que será el extremo de la nueva diagonal.
- `buscarDiagonal(Poli, indices)`: recorre los vértices del subpolígono hasta encontrar uno convexo, y aplica `esOreja` para determinar la diagonal a trazar. Gestiona el caso degenerado de vértices colineales (retornando `NULL`).
- `triangulacion_rec(Poli, indices)`: función principal recursiva que implementa el algoritmo completo, devolviendo una lista de triángulos (cada uno representado por tres índices de columna de la matriz `Poli`).

3.3. Casos de prueba y resultados

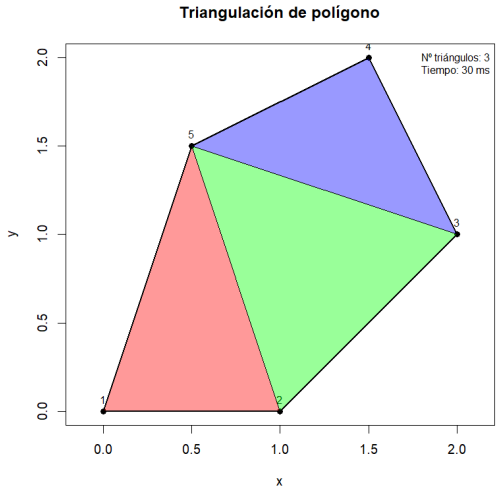
Se han diseñado seis casos de prueba que cubren distintos tipos de polígonos: convexo simple, con concavidad pronunciada, con vértices casi colineales, con forma de aguja, tipo peine (de Chvátal) y una ameba irregular con 100 vértices.

Los resultados obtenidos se resumen en la Tabla 3.1.

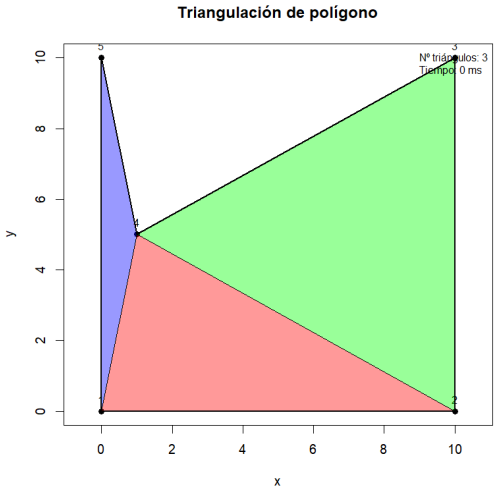
Polígono	n **vértices**	**Triángulos** $(n-2)$	**Tiempo (ms)**
Convexo simple	5	3	0
Concavidad boca	5	3	0
Pentágono, casi cuadrado	5	3	280
Caso límite	5	3	0
Peine de Chvátal	32	14	20
Ameba (100 pts.)	100	98	350

Tabla 3.1: Resumen de resultados de la triangulación para cada caso de prueba.

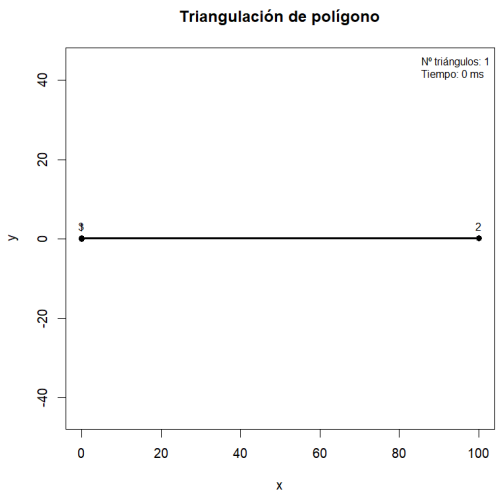
En todos los casos el número de triángulos obtenido coincide con el valor teórico $n - 2$. El tiempo de ejecución crece con la complejidad del polígono, siendo el caso de la ameba el más costoso debido a su gran número de vértices



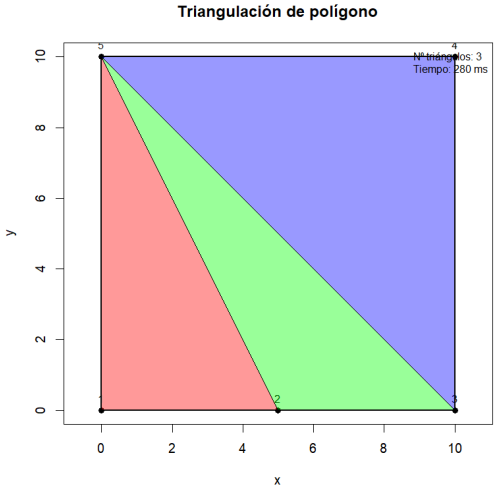
(a) Polígono convexo simple (5 vértices, 3 triángulos).



(b) Polígono con concavidad profunda (*boca*).

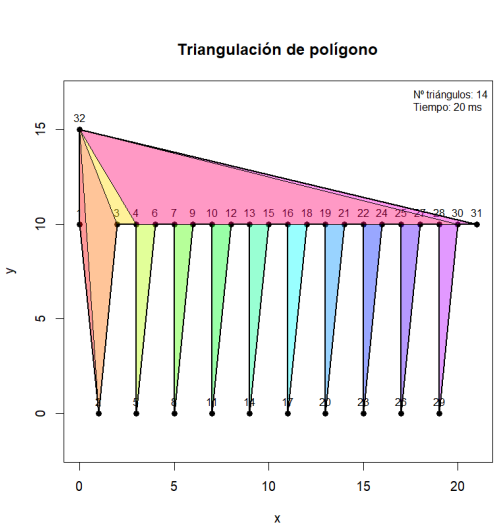


(c) Polígono con vértice casi colineal (caso límite).

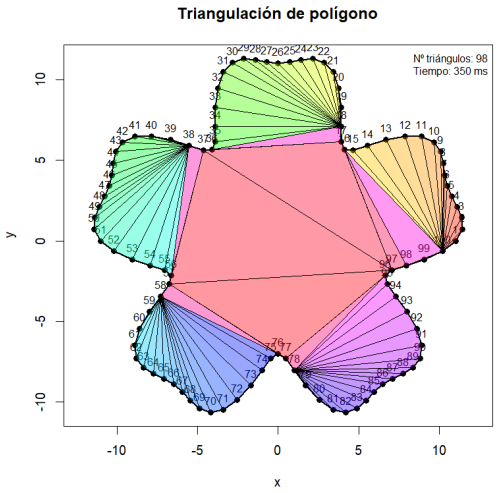


(d) Pentágono cercano a un cuadrado, con pequeña desalineación en su lado inferior, triangulado (3 triángulos).

Figura 3.1: Primeros casos de prueba de triangulación. Cada triángulo se representa con un color distinto.



(a) Peine de Chvátal[4] (14 triángulos).



(b) Ameba irregular con 100 vértices (98 triángulos).

Figura 3.2: Casos de prueba avanzados de triangulación.

y a la irregularidad de su contorno, que obliga al algoritmo a buscar diagonales no triviales en múltiples pasos recursivos. Cabe destacar el pico en el caso del pentágono, con 280 ms, que posiblemente se deba simplemente a alguna acción secundaria arbitraria del ordenador, y posiblemente dejaría de ser un caso notorio si se repitiese el algoritmo más veces y se hiciera la media.

3.4. Consideraciones sobre casos especiales

El algoritmo incorpora tratamiento explícito para los siguientes casos degenerados:

- **Vértices colineales:** Las funciones de comparación (`negativo_tolerancia`, `positivo_tolerancia`, `nulo_tolerancia`) usan una tolerancia de 10^{-8} para evitar errores numéricos de punto flotante. El polígono tipo aguja (tres puntos casi colineales) y el pentágono caso límite (vértice casi colineal con los dos adyacentes) se gestionan correctamente.
- **Polígono de tipo peine:** El peine de Chvátal es el caso canónico en el que se necesita el número máximo de vigilantes ($\lfloor n/3 \rfloor$) para cubrir la galería. El algoritmo lo triangula correctamente en $n - 2$ triángulos.
- **Vértices duplicados:** La función `triangulacion_rec` elimina puntos duplicados al inicio mediante `unique(..., MARGIN=2)`.

4

Parte II: Coloración de Grafos

4.1. Fundamentos teóricos

Dado un grafo simple $G = (V, E)$, una **coloración propia** de sus vértices es una asignación $c : V \to \{1, 2, \ldots, k\}$ tal que $c(u) \neq c(v)$ para todo par de vértices adyacentes $(u, v) \in E$. El mínimo k para el que existe tal coloración se denomina **número cromático** de G y se denota $\chi(G)$ [2].

Determinar $\chi(G)$ es un problema NP-completo en general. Sin embargo, existen heurísticas que producen coloraciones con un número de colores reducido en tiempo polinomial. En este trabajo se implementa el algoritmo **DSATUR** [5], que es una de las heurísticas más eficaces conocidas.

La idea central de DSATUR es la de **saturación**: la saturación de un vértice v no coloreado es el número de colores distintos ya asignados a sus vecinos. En cada paso, se elige el vértice no coloreado con mayor saturación (en caso de empate, se desempata por grado). A continuación, se le asigna el menor color disponible (no usado por ningún vecino).

Esta estrategia es exacta para grafos bipartitos (produce siempre una 2-coloración) y produce la coloración óptima para grafos ciclo y grafos completos.

4.2. Descripción del algoritmo

La implementación en `R` se organiza en tres funciones principales:

- `matriz_grafo(nVertices, aristas)`: construye la matriz de adyacencia $A \in \{0,1\}^{n \times n}$ del grafo a partir de su lista de aristas.
- `saturacion(i, matriz, coloreados)`: calcula la saturación del vértice i, contando el número de colores distintos (distintos de 0) presentes en sus vecinos según el vector `coloreados`.
- `coloracion(nVertices, aristas, coloreados)`: función principal que implementa el bucle de *DSATUR*. En cada iteración, calcula las saturaciones de todos los vértices no coloreados, elige el de mayor saturación (desempatando por grado) y le asigna el menor color disponible.

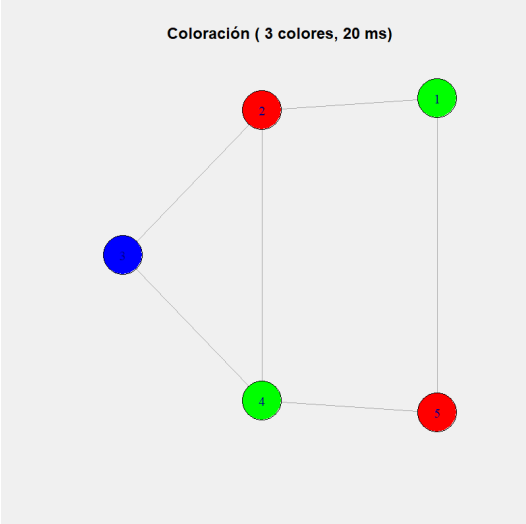
4.3. Grafos de prueba

Se han definido cinco familias de grafos para validar el algoritmo, más un grafo ad hoc adicional:

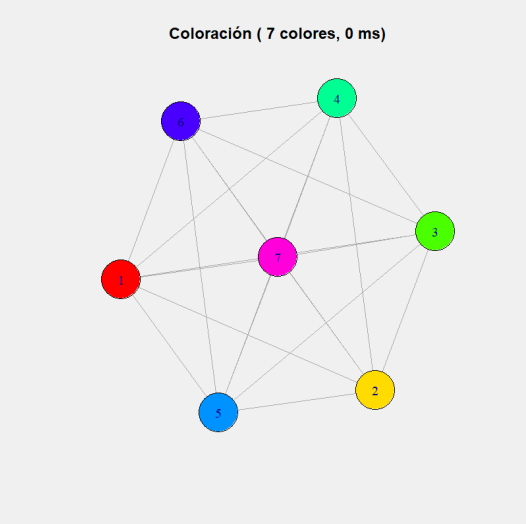
- **Grafo general** (`aristas1`): 5 vértices con 6 aristas, incluyendo una cuerda interior. Se espera $\chi = 3$.
- **Grafo completo** K_7 (`aristas2`): grafo con todos los pares de vértices conectados. Se espera $\chi = 7$.
- **Grafo ciclo** C_7 (`aristas3`): ciclo de 7 vértices. Al ser impar, $\chi = 3$.
- **Grafo bipartito** $K_{4,5}$ (`aristas4`): grafo bipartito completo. Se espera $\chi = 2$.
- **Grafo rueda** W_8 (`aristas5`): vértice central conectado a todos los de un ciclo de 7. Se espera $\chi = 4$ (el ciclo es impar).
- **Grafo adicional** (`aristas6`): 10 vértices que forman dos ciclos de 5 conectados entre sí. Se espera $\chi = 3$.

4.4. Resultados

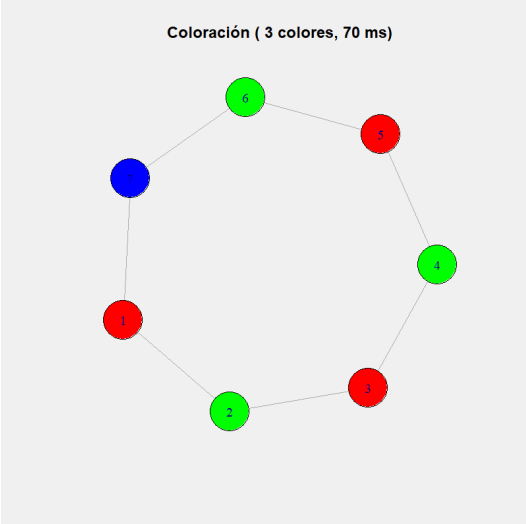
Los grafos han sido representados con la librería `igraph`. Los resultados se resumen en la Tabla 4.1. En todos los casos el número de colores obtenido por DSATUR coincide con el número cromático teórico $\chi(G)$.



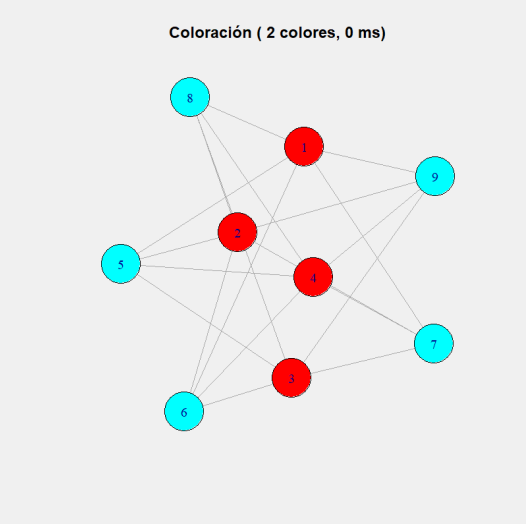
(a) Grafo general (5 vértices): 3 colores.



(b) Grafo completo K_7: 7 colores.



(c) Ciclo C_7: 3 colores.

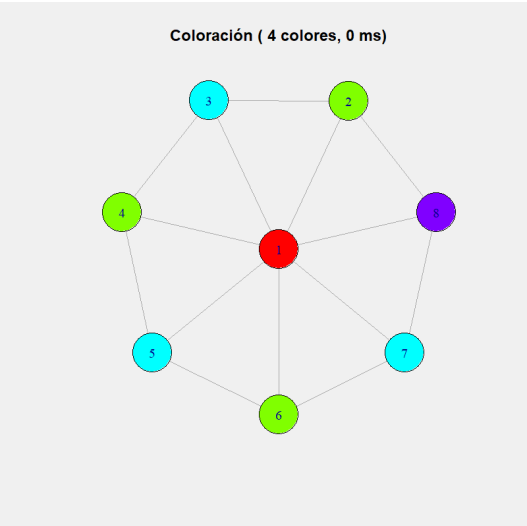


(d) Grafo bipartito $K_{4,5}$: 2 colores.

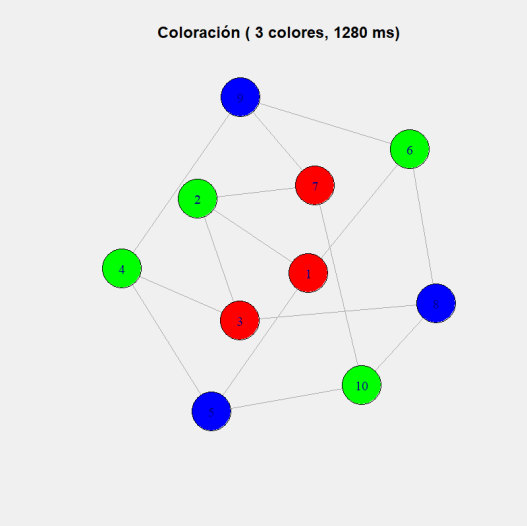
Figura 4.1: Coloraciones obtenidas para los primeros cuatro grafos de prueba.

Grafo	$\|V\|$	$\|E\|$	χ **teórico**	**Colores DSATUR**
General (aristas1)	5	6	3	3
Completo K_7	7	21	7	7
Ciclo C_7	7	7	3	3
Bipartito $K_{4,5}$	9	20	2	2
Rueda W_8	8	14	4	4
Ejemplo adicional	10	15	3	3

Tabla 4.1: Resultados de coloración para cada grafo de prueba.



(a) Grafo rueda W_8: 4 colores.



(b) Grafo adicional (10 vértices): 3 colores.

Figura 4.2: Coloraciones obtenidas para los dos grafos de prueba restantes.

5

Resultados y Discusión

5.1. Triangulación

El algoritmo de triangulación por eliminación de orejas produce resultados correctos en todos los casos de prueba considerados, respetando el invariante teórico $n-2$ triángulos para polígonos de n vértices. La gestión de casos degenerados mediante tolerancias numéricas resulta fundamental para la robustez del algoritmo: sin ella, los casos límite (vértice casi colineal, polígono tipo aguja) producirían errores numéricos o bucles infinitos.

El caso más interesante es el del peine de Chvátal: al ser un polígono con muchos vértices reflex alineados en la base, el algoritmo debe recorrer un número considerable de vértices antes de encontrar uno convexo que sea oreja, lo que incrementa el tiempo de ejecución respecto a un polígono convexo de igual número de vértices.

El caso de la ameba con 100 vértices demuestra que el algoritmo escala correctamente a polígonos de tamaño moderado, produciendo 98 triángulos en aproximadamente 350 ms.

5.2. Coloración de grafos

DSATUR produce la coloración óptima (número cromático exacto) en todos los casos de prueba. Esto es esperable para los grafos estudiados:

- El grafo completo K_n siempre requiere n colores, y DSATUR lo detecta correctamente al saturar en $k-1$ colores cada nuevo vértice de grado máximo.
- El grafo bipartito $K_{4,5}$ es 2-coloreable por definición; DSATUR lo colorea en 2 colores.
- El ciclo impar C_7 requiere 3 colores; DSATUR los asigna correctamente.
- La rueda W_8 tiene el vértice central adyacente a todos los del ciclo impar, lo que fuerza el cuarto color.

Los tiempos de ejecución de la coloración son muy reducidos para los grafos considerados (inferiores a 1,3), lo que confirma la eficiencia práctica de DSATUR para grafos de tamaño moderado.

5.3. Aplicación a un mapa geográfico

Descripción del mapa y la coloración

Se han seleccionado dos mapas geográficos reales para ilustrar la aplicación del algoritmo DSATUR a instancias de mayor escala y relevancia práctica: el mapa de las **provincias de China** y el mapa de los **estados de Estados Unidos**.

En ambos casos, el mapa se ha codificado como un grafo de la siguiente manera:

- Cada región (provincia o estado) se representa como un **vértice** del grafo.
- Dos vértices están conectados por una **arista** si y solo si las regiones correspondientes comparten una frontera terrestre común (se excluyen las fronteras puntuales).

Para manejar la información sobre las coordenadas geográficas reales de las fronteras políticas, se ha hecho uso de la librería `rnaturalearth` y `rnaturalearthdata`, y para tener los datos en un formato mejor se ha usado la librería `sf`.

El algoritmo DSATUR ha producido en ambos casos una coloración válida utilizando **4 colores**, lo cual es coherente con el célebre **Teorema de los Cuatro Colores**: todo mapa planar puede colorearse con a lo sumo 4 colores de forma que ningún par de regiones adyacentes comparta color.

Visualización

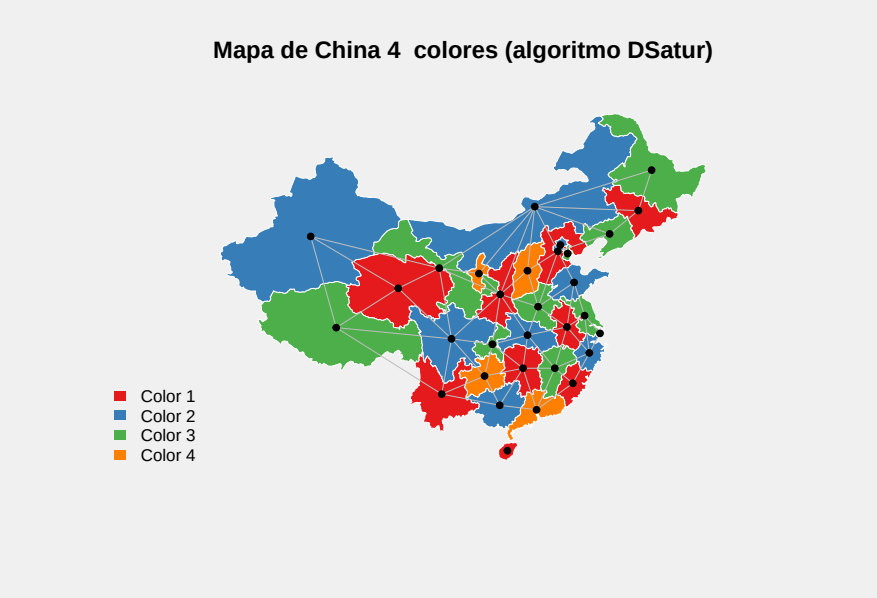


Figura 5.1: Coloración del mapa de China obtenida con DSATUR. Se han empleado 4 colores.

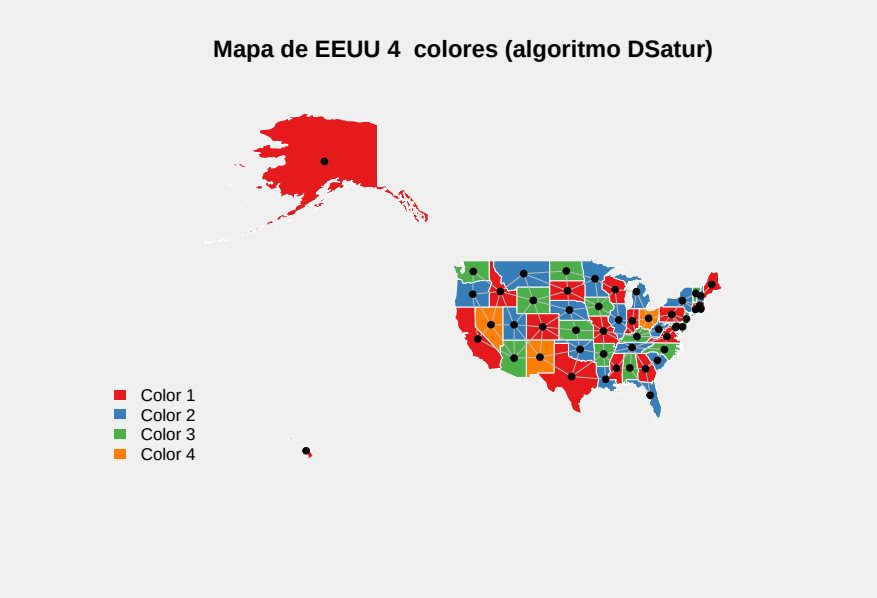


Figura 5.2: Coloración del mapa de Estados Unidos obtenida con DSATUR. Se han empleado 4 colores.

Análisis de los resultados

En los dos mapas, DSATUR ha necesitado exactamente **4 colores** para obtener una coloración propia. Este resultado confirma que ambos grafos de adyacencia son 4-coloreables, en concordancia con el Teorema de los Cuatro Colores (Appel y Haken, 1976 [6]), que establece que todo grafo planar es 4-coloreable.

Este resultado ilustra una diferencia importante respecto a los grafos de prueba del capítulo anterior: mientras que para grafos como K_7 o W_8 el número cromático podía ser mayor que 4, los grafos de adyacencia de mapas geográficos son siempre grafos planares, y el Teorema de los Cuatro Colores garantiza que 4 colores son siempre suficientes. DSATUR, aplicado sobre estas instancias reales, ha encontrado dicha coloración de forma eficiente.

6

Conclusiones y Trabajos Futuros

6.1. Conclusiones

- **Corrección del algoritmo de triangulación:** El algoritmo recursivo de triangulación produce triangulaciones válidas para todos los polígonos simples probados, incluyendo casos degenerados, validando así la implementación del Lema de Meisters. El número de triángulos obtenido coincide siempre con el valor teórico $n - 2$.
- **Corrección del algoritmo de coloración:** DSATUR produce el número cromático exacto para todos los grafos de prueba considerados. Su estrategia de priorización por saturación es especialmente eficaz para grafos con estructura regular (completos, ciclos, bipartitos, ruedas).
- **Robustez numérica:** El uso de funciones de comparación con tolerancia (10^{-8}) es esencial para la correcta gestión de casos casi degenerados, evitando errores en la clasificación de vértices convexos o en la detección de puntos interiores.

6.2. Trabajos futuros

- **Triangulación de Delaunay:** Implementar la triangulación de Delaunay, que maximiza el ángulo mínimo de los triángulos y es preferible en aplicaciones numéricas (elementos finitos, interpolación).

- **Extensión a polígonos con huecos:** El algoritmo actual solo maneja polígonos simples. Extenderlo para tratar polígonos con agujeros interiores (polígonos multiconexos) ampliaría considerablemente su aplicabilidad.
- **Coloración exacta por backtracking:** Para grafos pequeños, implementar un algoritmo de coloración exacta por ramificación y poda permitiría comparar la solución de DSATUR con el óptimo global y evaluar la calidad de la heurística.
- **Aplicación completa al Problema de las Galerías:** Implementar al completo la triangulación, 3-coloración y selección de vigilantes, y visualizar gráficamente los vigilantes óptimos sobre el polígono original.

Bibliografía

[1] G. H. Meisters. *Polygons have ears*. American Mathematical Monthly, 82(6):648–651, 1975.

[2] C. Simón de Blas. *Práctica 9: Triangulaciones*. Apuntes de Geometría Computacional, Universidad Rey Juan Carlos, 2025.

[3] Centro de Investigación en Matemáticas. *Triangulación de Polígonos*. `https://www.cimat.mx/~alram/analisis_algo/05_TriangulacionPoligonos.pdf`, s.f.

[4] V. Chvátal. *A combinatorial theorem in plane geometry*. Journal of Combinatorial Theory, Series B, 18(1):39–41, 1975.

[5] R. M. R. Lewis. *A Guide to Graph Colouring: Algorithms and Applications*. Springer International Publishing, Cham, Suiza, 2016.

[6] K. Appel y W. Haken. *Every planar map is four colorable*. Bulletin of the American Mathematical Mathematical Society, 82(5):741–743, 1976.